Four-Port Switch Project

Implementation, Verification, and Synthesis

תוכן

3 ..... Introduction
4 ..... Code Files Description
11 ..... Verification Environment
16 ..... Synthesis & Performance Analysis
24 ..... Coverage Appendix

Introduction

Project Overview & Goals

The objective of this project was to design, implement, and verify a four-port packet switch capable of receiving data packets from any input port and forwarding them to one or more designated destination ports in real-time. The switch was designed to meet strict data integrity requirements, ensuring zero data loss while supporting three primary traffic types:

- **Unicast:** Forwarding a packet to a single destination port.
- **Multicast:** Forwarding a packet to multiple destination ports simultaneously.
- **Broadcast:** Distributing a packet to all output ports in the system.

The design adheres to specific routing rules, ensuring that packets are not routed back to their source port (except in broadcast scenarios), that each destination receives exactly one copy, and that the system effectively handles traffic congestion using a fair **Round-Robin Arbitration** scheme.

Project Execution Flow

The project was executed in three main stages, simulating a full standard chip development flow:

1. **Stage A (RTL Design):** Design and implementation of the logic using SystemVerilog, including Finite State Machine (FSM) definitions, memory management (FIFOs), and routing architecture.
2. **Stage B (Verification):** Development of a modular, Object-Oriented (OOP) verification environment using Constrained-Random Verification (CRV) to ensure 100% functional coverage and validation against a Reference Model.
3. **Stage C (Synthesis & Analysis):** Logical synthesis of the design, performance analysis (Area, Power, Frequency), and validation of Timing Constraints.

Code Files Description

This chapter details all source code files developed and utilized throughout the project, categorized into RTL Design files and Verification Environment files.

RTL Design Files The following files constitute the hardware implementation (Device Under Test - DUT):

File Name	Description
switch_4port.sv	Top Module: The main module connecting all four switch_port instances to the central Routing block.
switch_port.sv	Input Interface Module: Manages the input for each port. It instantiates the FIFO for buffering and the Master_Control_unit for flow control.
Routing.sv	Switch Fabric: Instantiates 4 arbiter blocks (one per output) to manage port contention. It implements the Transpose Logic, mapping slave grant signals (rdy_out_slave) back to the master's acknowledgment vector (sent_mask_m) to indicate successful routing.
arbiter.sv	Output Port Controller: Manages access to a specific output port. It wraps the Control Unit (Arbitration) and Data Path (Muxing) logic.
Arbiter_control_unit.sv	Arbitration Logic: Implements the Round-Robin algorithm to select a winning Master. It manages the grant signals and creates the round_robin_select pointer.
Arbiter_data_path.sv	Data Multiplexer: The routing matrix execution unit. It multiplexes the data, source, and target signals from the winning Master to the output port based on the selector from the control unit.
Master_control_unit.sv	Input FSM: The Finite State Machine managing packet progression. It transitions between IDLE, PACKET_RECEIVED, and CHECK_IF_ALL_PACKET_PASSED to ensure multicast packets are delivered to all targets before clearing the FIFO.
FIFO.sv	Data Buffer: A synchronous FIFO memory that stores packet_t structures. It manages full and empty pointers to handle bursty traffic.
internal_if.sv	Interface Definition: Defines the internal bus protocol connecting the ports. Includes signals: valid, source, target, and data with Master/Slave modports.
packet_types.sv	Global Types: A package file defining the packet_t struct used across the design to ensure consistent data widths and structure.
d_ff.sv	Basic Sequential Component: A wrapper module for D-Flip-Flops, used to implement registers and state memory throughout the design hierarchies.
tb.sv	Manual QA

Verification Environment Files The following files constitute the Testbench environment:

File Name	Description
switch_test.sv	Top-Level Testbench: The static container of the verification environment. It instantiates the DUT (switch_4port) and the Interface (port_if), generates the clock, and manages the simulation flow (initial block). Crucially, it defines and samples the Functional Coverage groups (cg_fsm_states, cg_global).
packet_pkg.sv	Verification Package: Aggregates all verification classes (Transaction, Driver, Monitor, Agent, etc.) into a single package (packet_pkg) to streamline compilation and scope management.
packet_data.sv	Transaction Object: Defines the packet class with properties (Source, Target, Data). It includes random constraints (c_legal_target, c_no_loopback) to generate valid Unicast, Multicast, and Broadcast traffic.

packet_vc.sv	Environment Wrapper: The top-level class-based component for a specific port. It instantiates the agent and manages the mailboxes connecting the monitors to the central checker.
agent.sv	Agent Container: A hierarchical unit that encapsulates the sequencer, driver, and monitor for a specific port. It handles the configuration and connection of these sub-components.
sequencer.sv	Stimulus Generator: Responsible for creating packet objects. It randomizes the packets based on the specific port_id (ensuring correct Source ID) and passes them to the Driver via a mailbox.
driver.sv	BFM (Bus Functional Model): Pulls packet transactions from the mailbox and drives them onto the physical pins of the Virtual Interface (vif) according to the required timing protocols.
monitor.sv	Passive Observer: Samples signals from the interface, reconstructs the packet transaction, and forwards it to the Checker/Scoreboard for validation via mailboxes.
checker.sv	Scoreboard & Reference Model: The central validation unit. It contains a Reference Model (expected_collector) to predict routing logic and a Scoreboard (check_out_port) to compare actual DUT outputs against the expected results.
component_base.sv	Base Class: A generic parent class inherited by all verification components. It provides naming mechanisms and hierarchical structure handling.
interface.sv	Virtual Interface: Defines the pin-level signals (valid_in, data_in, etc.) connecting the Testbench to the DUT ports. (Referred to as port_if in the Top Level).
debug_if.sv	Debug Interface: A white-box interface used to expose internal DUT signals (like full and wr_en from the FIFO) to the verification environment for assertion checking and coverage sampling.

Design

Introduction

Project Goal

The goal of this block is to implement a four-port packet switch whose primary function is to receive packets from any input port and forward them to one or more designated output ports. It adheres to strict routing rules, ensuring packets are not routed back to their source (unless broadcast), destinations receive exactly one copy, and no data corruption occurs.

High-Level Description

The four-port switch receives packets on its input ports and forwards them based on encoded routing information. It supports unicast, multicast, and broadcast behavior while guaranteeing that packets are not dropped or duplicated once accepted. The switch handles concurrent packet arrivals under high traffic load, maintaining predictable and deterministic routing.

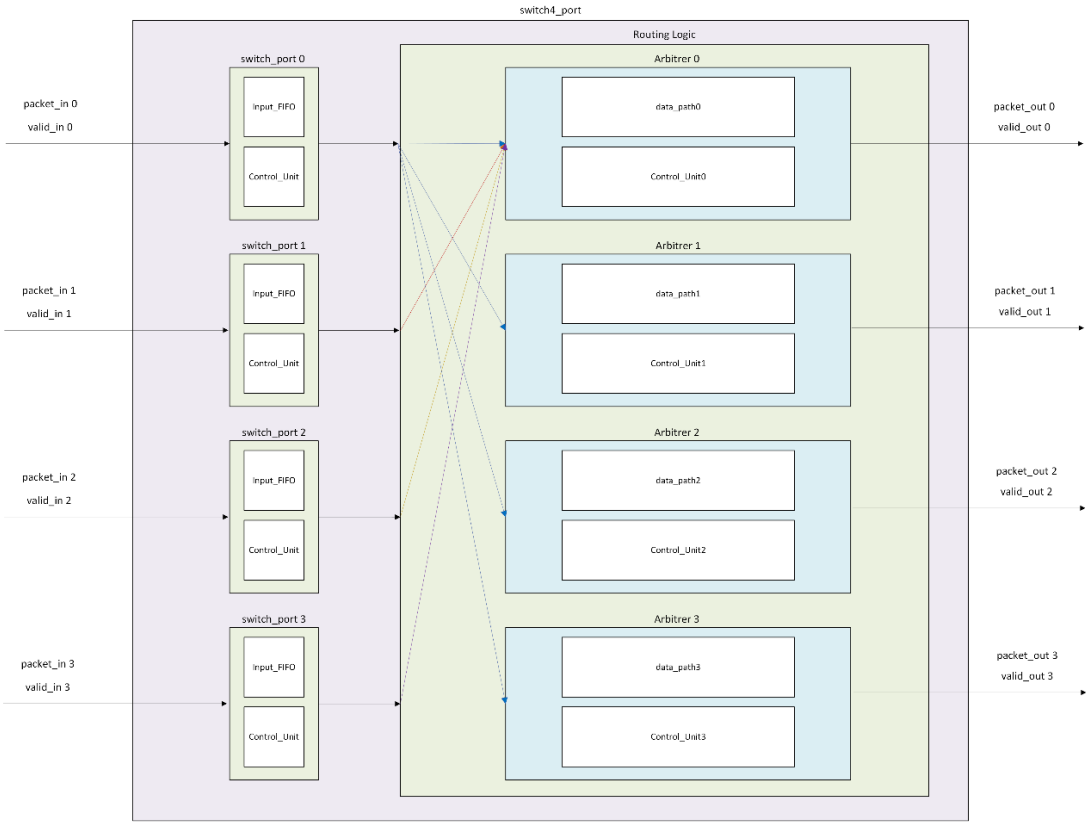
The block must comply with a defined set of routing rules that govern when a packet may or may not be forwarded to a given output port:

- A packet is not allowed to be routed back to the port from which it originated, unless it is explicitly a broadcast packet.
- Each destination must receive exactly one copy of the packet.
- The block must guarantee that every packet is forwarded exactly as it was received, without any modification or corruption of its source, target, or data fields.
- The block must guarantee that packets are not dropped, overwritten, or lost once accepted.

RTL Design and Architecture

Internal Architecture Overview

The switch consists of four independent switch_port input units and a centralized routing block composed of four arbiters. A switch_port stores packets in a local FIFO and forwards them to the routing stage, governed by a Master Control Unit FSM. The routing block directs packets to the appropriate slave port using a Round-Robin arbitration scheme.



External Interface Specification

This section defines all external interfaces of the switch block. The switch exposes four identical external ports sharing a common system clock and synchronous active-low reset.

Input Signals Description Table

Interface Signals (per port)

Signal Name	Width	Direction	Description
Valid_in	1	input	Indicates that the input port provides a valid packet on this cycle
Source_in	4	Input	One-hot encoded source port identifier
Target_in	4	input	Target bus specifying the destination output ports

Data_in	8	input	Packet payload data
Valid_out	1	output	Asserted when a valid packet is driven toward the corresponding output port
Source_out	4	output	Source field of the forwarded packet
Target_out	4	output	Final (possibly masked) target mask of the forwarded packet
Data_out	8	output	Forwarded packet payload data

Block input signals

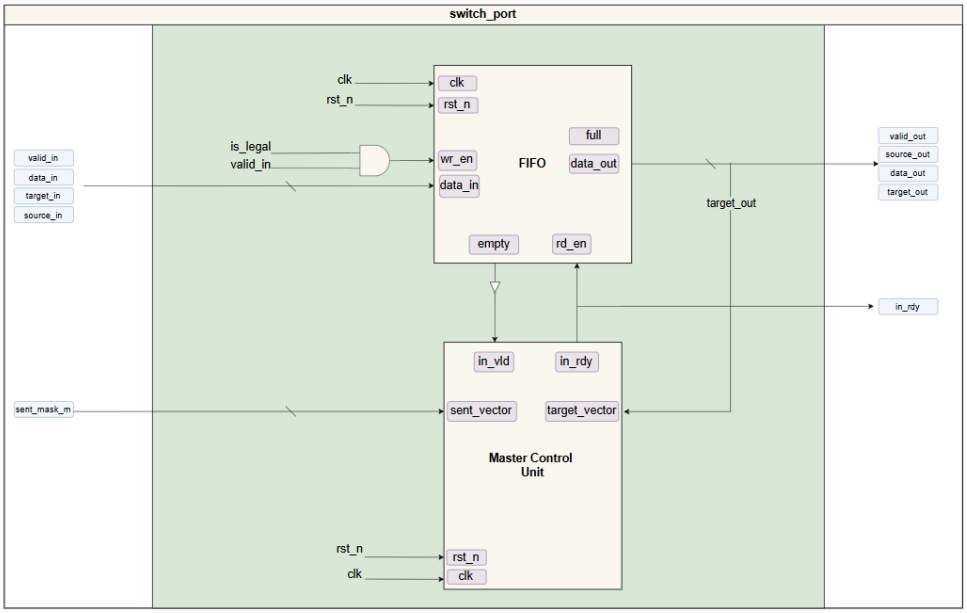
Signal Name	Width	Direction	Description
Clk	1	input	System clock used for all sequential logic
Rst_n	1	input	Synchronous active-low reset. It initializes all internal registers and FIFOs

Module-Level Descriptions

Switch Port & FIFO Logic

The switch_port unit acts as the ingress interface.

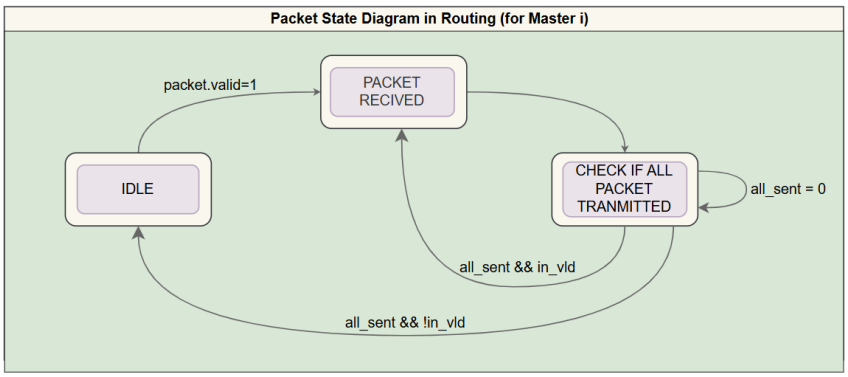
- **Input Side:** Controlled by FIFO capacity and packet validity. A dedicated function, is_packet_legal(), filters invalid packets before they enter the buffer.
- **Output Side:** Controlled by the Master Control Unit. The FSM generates a read enable signal only when the Routing block is ready (in_rdy).
- **Feedback Mechanism:** The module receives a sent_mask_m signal from the central routing logic. This vector indicates which output ports have successfully consumed the current packet, allowing the FSM to track progress and discard the packet only when fully served.



Master Control FSM

The Master Control Unit manages the packet's progression. It tracks whether the current packet has been fully served to all target destinations.

The FSM ensures in_rdy is high only when the packet is fully transmitted or the unit is IDLE. The FSM transitions between three main states: **IDLE** (waiting for valid packet), **PACKET_RECEIVED** (processing header and routing), and **CHECK_IF_ALL_PACKET_TRANSMITTED** (ensuring delivery to all multicast targets before clearing the buffer)

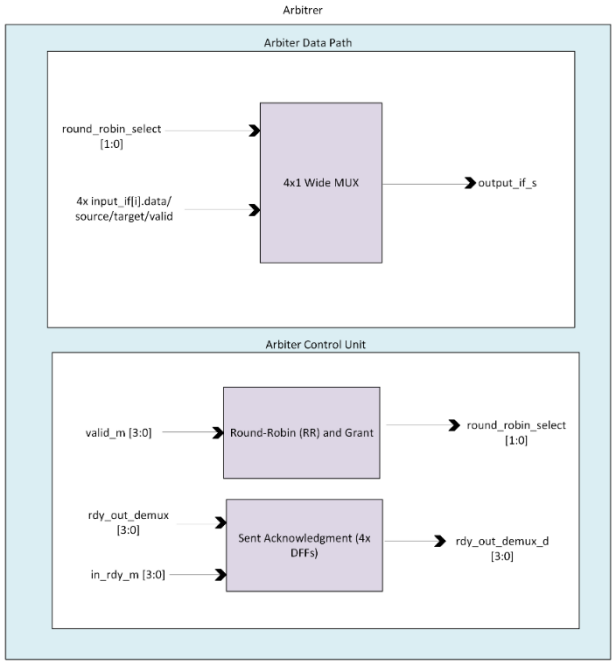


Routing & Arbiter Control

The Routing block contains four independent arbiters. Each arbiter implements a robust scheme to handle contention:

- **Per-Master Internal Valid:** The arbiter maintains an internal valid bit for each master. This bit remains high only if the master requests this specific slave AND the packet has not yet been served.

- **Round-Robin Arbitration:** The grant signal is updated using a circular search logic starting from the previous winner. This ensures fairness and prevents starvation.



- **Sent Acknowledgments (DFFs):** Dedicated D-Flip-Flops track the service status. When a master is served, the corresponding DFF is set to '1', immediately clearing the master's internal valid bit to prevent duplicate transmission.

Handshake Protocols

The design supports a valid/ready handshake. However, to simplify the implementation, the ready signal from each slave is tied to logic 1 (always ready), removing the need for output FIFOs.

Error Handling

Illegal packets (e.g., non-one-hot source) are filtered at the input stage by the is_packet_legal() function. These packets are not written to the FIFO, preventing propagation through the router.

Verification Environment

Verification Methodology & Goals

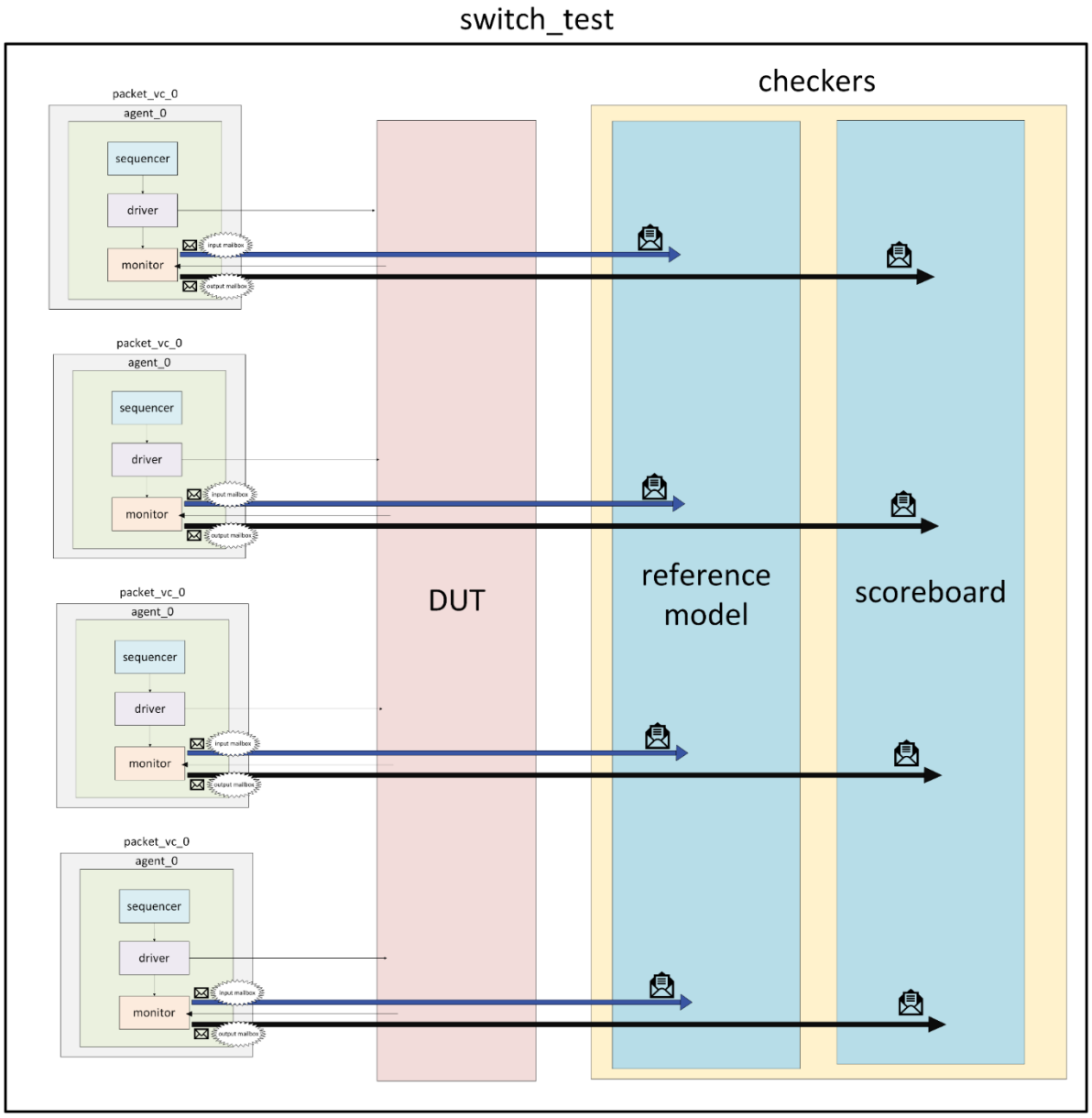
The verification strategy is based on Constrained-Random Verification (CRV) using a modular, object-oriented approach in SystemVerilog. The environment was built from scratch without external libraries (like UVM) to validate routing logic, concurrency, and data integrity. Key verification goals included:

- **Routing Logic:** Ensuring packets reach all intended destinations (Unicast, Multicast, Broadcast) without corruption.
- **Integrity:** Confirming zero packet loss or duplication under heavy traffic.
- **Concurrency:** Verifying Round-Robin arbitration during simultaneous arrivals.

Verification Architecture

The environment employs a layered architecture using virtual interfaces to abstract hardware details.

- **Packet Model**: A transaction class with randomizable fields (Source, Target, Data) and validity constraints.
- **Sequencer & Driver**: Generates packet sequences and drives pin-level signals via the virtual interface using a Valid/Ready handshake.
- **Monitor**: Passively observes input/output ports and forwards transactions to the Scoreboard via Mailboxes.
- **Agent:** A hierarchical container grouping the sequencer, driver, and monitor per port.



Automated Checker (Scoreboard)

The Checker is the heart of the validation process. It includes a **Reference Model** that predicts expected routing based on switch logic. The Self-Checking Scoreboard compares actual DUT outputs against these predictions.

- **Reference Comparison:** For every packet entering the DUT, the Scoreboard predicts the expected output.
- **Data Integrity:** It compares the actual data, source, and target fields captured at the output ports against the predicted values.

Constrained-Random Implementation

To stress the design, robust constraints were implemented in the packet class:

- **Traffic Distribution:** 40% Single, 40% Multicast, and 20% Broadcast to exercise complex replication logic.
- **Loopback Prevention:** A critical constraint ensuring a packet is never routed back to its source port (Source & Target != 0).
- **Randomized Inter-Packet Gap (IPG):** The driver injects random idle cycles to test the FSM's ability to handle "back-to-back" bursts.

Functional Coverage Analysis

A multi-layered coverage model was implemented to achieve 100% functional coverage:

- **Ingress Coverage (cg_in_cov):** Verifies traffic diversity, including cp_src_match_port (integrity check) and cp_mcast_mask (all multicast combinations).
- **Egress Coverage (cg_out_cov):** Monitors output behavior, specifically cp_b2b which tracks "Back-to-Back" delivery to verify maximum throughput.
- **FSM Coverage (cg_fsm_states):** Ensures the Master Control Unit visited all legal states (IDLE, PACKET_RECEIVED, CHECK_IF_ALL_PACKET_TRANSMITTED) and transitions.
- **Global System Logic (cg_global_edge):** Tracks system-wide events like Ingress Contention (multiple requests) and Parallel Output Processing.

(full coverage result in the appendix)

Simulation Results & Protocol Validation

To ensure the robustness of the 4-Port Switch, the verification environment executed a diverse set of test cases designed to stress the RTL logic.

Functional Assertions & Protocol Validation

Beyond the Scoreboard's data comparison, real-time Functional Assertions were implemented within the Monitor and the Testbench top-level to ensure strict protocol adherence and internal integrity at every clock cycle:

- **Signal Integrity (!$isunknown):** Monitors the output bus to verify that source, target, and data signals do not contain 'X' (unknown) or 'Z' (high-impedance) values when valid_out is asserted. This guarantees clean digital transitions throughout the simulation.
- **Architectural Legality:** The testbench utilizes the is_packet_legal function to validate every outbound packet. This assertion confirms that the switch only routes packets that follow the

predefined protocol rules (e.g., valid source-target combinations and One-Hot encoding constraints).

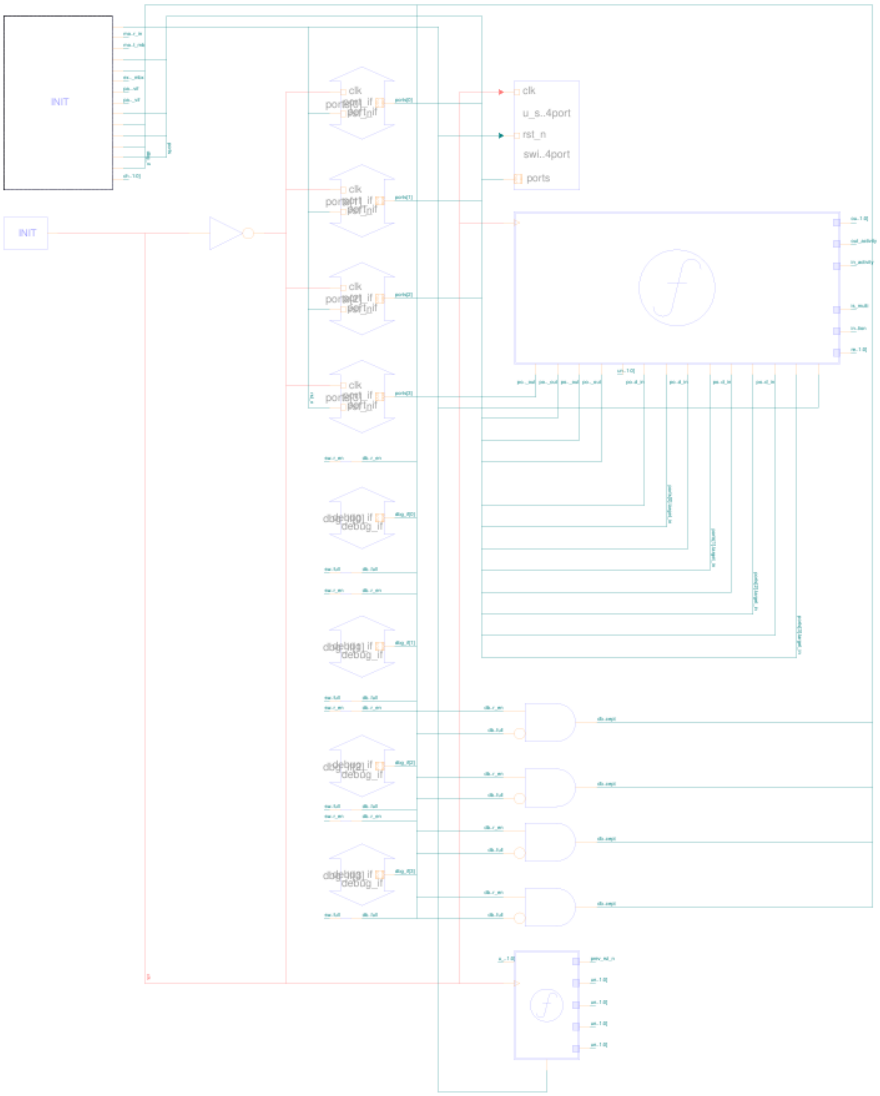
- **Routing Alignment Check:** A critical assertion verifies that if a packet physically appears on Port [n], the target_out[n] bit in the destination bitmap must be set. This ensures that the internal routing matrix correctly delivered the packet to the physical port specified in the header.
- **FIFO Integrity Monitoring:** By utilizing the white-box debug_if interface, the environment asserts that no Overflow or Underflow events occur during burst traffic. It verifies that write operations are never accepted when the internal full signal is asserted.
- **Arbiter Fairness Verification:** The verification logic includes checks to ensure the Round-Robin arbiter does not permanently starve any requestor. It validates that valid requests eventually receive a grant, maintaining the fairness guarantees of the architecture.

Final Scoreboard Status:

As shown in the logs, the environment reports a "**MATCH**" for every successful routing. The simulation concluded with **0 Errors**, confirming data integrity and routing correctness.

The full output log is attached in a separate file.

```
[checker] OUT0: exp_added=210 act_seen=210 pass=210 fail=0 exp_left=0
[checker] OUT1: exp_added=204 act_seen=204 pass=204 fail=0 exp_left=0
[checker] OUT2: exp_added=197 act_seen=197 pass=197 fail=0 exp_left=0
[checker] OUT3: exp_added=214 act_seen=214 pass=214 fail=0 exp_left=0
$finish called from file "switch_test.sv", line 161.
$finish at simulation time              5025000
           V C S   S i m u l a t i o n   R e p o r t
Time: 5025000 ps
CPU Time:      0.320 seconds;       Data structure size:   0.1Mb
Sun Jan 18 11:17:50 2026
#qrsh -V -cwd -b y -q sim_normal ./simv +ntb_random_seed_automatic 2>&1 | tee log
```



Design Name	switch_test		
Designer	royfriedman		
Description			
Date	Jan 20, 2026	Version	

Synthesis & Performance Analysis

Synthesis Methodology & Constraints

The RTL design was synthesized using **Synopsys Fusion Compiler (FC)** . The synthesis process aimed to optimize the design for Area, Power, and Timing under strict constraints.

- **Target Frequency:** 100 MHz (Period: 10 ns).
- **Optimization Goal:** Minimize Area and Power while meeting both Setup and Hold timing constraints.

Baseline Synthesis Results (Without Clock Gating)

In the first synthesis run, the design was mapped without any special power optimizations to establish a baseline.

```
                    Clock Gating Summary
 ------------------------------------------------------------
 |  Number of Clock gating elements          |            0       |
 |                                           |                    |
 |  Number of Tool-Inserted Clock gates      |         0   (0.00%) |
 |                                           |                    |
 |  Number of Pre-Existing Clock gates       |         0   (0.00%) |
 |                                           |                    |
 |  Number of Gated registers                |         0   (0.00%) |
 |                                           |                    |
 |  Number of Tool-Inserted Gated registers  |         0   (0.00%) |
 |                                           |                    |
 |  Number of Pre-Existing Gated registers   |         0   (0.00%) |
 |                                           |                    |
 |  Number of Ungated registers              |       368 (100.00%) |
 |                                           |                    |
 |  Total number of registers                |           368      |
 |                                           |                    |
 |  Max number of Clock Gate Levels          |            0       |
 ------------------------------------------------------------
```

- **Area:** Total cell area was **5626.24 [um^2]** (1299 cells).

```
****************************************

Report : area

Design : switch_4port

Version: V-2023.12-SP3

Date   : Sat Jan 17 21:07:00 2026

****************************************
```

```
Number of ports:                          2025

Number of nets:                           2936

Number of cells:                          1299

Number of combinational cells:             931

Number of sequential cells:                368

Number of macros/black boxes:                0

Number of buf/inv:                         115

Number of references:                       45
```

```
Number of references:                       45

Combinational area:                   2411.83

Buf/Inv area:                          173.07

Noncombinational area:                3214.41

Macro/Black Box area:                    0.00

Total cell area:                      5626.24
```

- **Timing:**

  Setup Slack: **2.66 ns** (Critical Path: 6.68 ns).

```
****************************************

Report : timing

-   path_type full

-   delay_type max

-   max_paths 1

-   report_by design

Design : switch_4port

Version: V-2023.12-SP3
```

```
clock clk (rise edge)
10.00     10.00

 clock network delay (ideal)
0.00     10.00


u_Routing/SLAVE_SWITCH[1].u_arbiter/u_Arbiter_c
ontrol_unit/SELECT_RDY[1].u_d_ff/dout_reg[0]/CL
K (SDFFARX1_HVT)

 10.00   0.00                          r

 clock uncertainty
-0.15      9.85
```

```
library setup time
-0.51      9.34

  data required time
9.34

  data required time
9.34

  data arrival time
-6.68

  slack (MET)
2.67
```

Hold Slack: 0.53 ns (Met), confirming no race conditions

```
****************************************

Report : timing

-   path_type full

-   delay_type min

-   max_paths 1

-   report_by design

Design : switch_4port

Version: V-2023.12-SP3
```

```
  clock clk (rise edge)
0.00      0.00

  clock network delay (ideal)
0.00      0.00


SWITCH_UNIT_MASTER_CONTROL[2].u_switch_port/u_F
IFO/data_out_reg[target][3]/CLK (SDFFX2_RVT)


0.00      0.00 r
```

```
library hold time
-0.10     -0.10

  data required time
-0.10

  data required time
-0.10

  data arrival time
-0.43

  slack (MET)
0.53
```

- **Power:** Total power consumption was approximately **~250 [uW]**, with the Clock Network consuming over 50% of the dynamic power.

```
Cell Internal Power    = 1.43e+08 pW(98.6% )

 Net Switching Power    = 2.10e+06 pW(1.4% )

Total Dynamic Power      = 1.45e+08 pW(100.0%)

Cell Leakage Power       = 1.06e+08 pW

Power Group         Internal Power        Switching Power          Leakage Power            Total Power    (   %  )    Attrs

--------------------------------------------------------------------------------------------------------------------

io_pad                    0.00e+00               0.00e+00               0.00e+00               0.00e+00    (0.0% )

memory                    0.00e+00               0.00e+00               0.00e+00               0.00e+00    (0.0% )

black_box                 0.00e+00               0.00e+00               0.00e+00               0.00e+00    (0.0% )

clock_network             1.29e+08               0.00e+00               0.00e+00               1.29e+08    ( 51.7%)        i

register                  1.16e+07               3.07e+05               6.99e+07               8.18e+07    (32.7% )

sequential                0.00e+00               0.00e+00               0.00e+00               0.00e+00    (0.0% )

combinational             1.87e+06               1.79e+06               3.56e+07               3.93e+07    (15.7% )

--------------------------------------------------------------------------------------------------------------------

Total                     1.43e+08 pW            2.10e+06 pW            1.06e+08 pW            2.50e+08 pW
```

Optimized Synthesis Results (With Clock Gating)

In the second run, automatic Clock Gating (CG) was enabled. The tool replaced standard registers with Integrated Clock Gating (ICG) cells, optimizing the clock tree.

```
                    Clock Gating Summary
-----------------------------------------------------------------
|  Number of Clock gating elements          |            23       |
|                                           |                     |
|  Number of Tool-Inserted Clock gates      |        23 (100.00%) |
|                                           |                     |
|  Number of Pre-Existing Clock gates       |         0   (0.00%) |
|                                           |                     |
|  Number of Gated registers                |       338  (91.85%) |
|                                           |                     |
|  Number of Tool-Inserted Gated registers  |       338  (91.85%) |
|                                           |                     |
|  Number of Pre-Existing Gated registers   |         0   (0.00%) |
|                                           |                     |
|  Number of Ungated registers              |        30   (8.15%) |
|                                           |                     |
|  Total number of registers                |           368       |
|                                           |                     |
|  Max number of Clock Gate Levels          |             1       |
-----------------------------------------------------------------
```

- **Area:** Total area dropped to **5019.09 [um^2]** (1051 cells).

```
****************************************

Report : area

Design : switch_4port

Version: V-2023.12-SP3

Date   : Sat Jan 17 20:57:04 2026

****************************************
```

```
Number of ports:                          2087

Number of nets:                           2735

Number of cells:                          1051

Number of combinational cells:             660

Number of sequential cells:                391

Number of macros/black boxes:                0

Number of buf/inv:                          82

Number of references:                       37
```

```
Combinational area:                    1684.47

Buf/Inv area:                           114.36

Noncombinational area:                 3334.62

Macro/Black Box area:                     0.00


Total cell area:                       5019.09
```

- **Timing:** Margins improved significantly due to reduced congestion:
  - **Setup Slack:** Increased to **3.81 ns** (Critical Path: 5.53 ns).

```
****************************************

Report : timing

        -path_type full

        -delay_type max

        -max_paths 1

        -report_by design

Design : switch_4port

Version: V-2023.12-SP3Date
```

```
clock clk (rise edge)
10.00     10.00

  clock network delay (ideal)
0.00     10.00


u_Routing/SLAVE_SWITCH[0].u_arbiter/u_Arbiter_c
ontrol_unit/SELECT_RDY[1].u_d_ff/dout_reg[0]/CL
K (SDFFARX1_HVT)


0.00     10.00 r

  clock uncertainty
-0.15      9.85
```

```
  data required time
9.34



  data required time
9.34

  data arrival time
-5.53

  slack (MET)
3.82
```

  - **Hold Slack:** Remained stable at **0.53 ns** (Met).

```
****************************************

Report : timing

        -path_type full

        -delay_type min

        -max_paths 1

        -report_by design

Design : switch_4port

Version: V-2023.12-SP3
```

```
clock clk (rise edge)
0.00      0.00

  clock network delay (ideal)
0.00      0.00


SWITCH_UNIT_MASTER_CONTROL[3].u_switch_port/u_F
IFO/data_out_reg[target][2]/CLK (SDFFX2_RVT)


0.00      0.00 r
```

```
library hold time
-0.10     -0.10

  data required time
-0.10

  data required time
-0.10

  data arrival time
-0.43

  slack (MET)
0.53
```

- **Power:** Total power dropped to **~134 [uW]**. The Clock Network power was slashed by **~80%** (down to 2.46 X 10^7 pW), proving the efficiency of the ICG insertion.

```
Cell Internal Power    = 3.77e+07 pW(92.6% )

 Net Switching Power    = 3.02e+06 pW(7.4% )

Total Dynamic Power      = 4.07e+07 pW(100.0%)

Cell Leakage Power       = 9.31e+07 pW

Power Group         Internal Power        Switching Power          Leakage Power           Total Power    (   %  )    Attrs

--------------------------------------------------------------------------------------------------------------------------------

io_pad                    0.00e+00              0.00e+00              0.00e+00              0.00e+00    (  0.0%)

memory                    0.00e+00              0.00e+00              0.00e+00              0.00e+00    (  0.0%)

black_box                 0.00e+00              0.00e+00              0.00e+00              0.00e+00    (  0.0%)

clock_network             2.22e+07              1.29e+06              1.09e+06              2.46e+07    ( 18.4%)        i

register                  1.39e+07              2.91e+05              6.94e+07              8.36e+07    ( 62.5%)

sequential                0.00e+00              0.00e+00              0.00e+00              0.00e+00    (  0.0%)

combinational             1.59e+06              1.44e+06              2.26e+07              2.56e+07    ( 19.2%)

--------------------------------------------------------------------------------------------------------------------------------

Total                     3.77e+07 pW           3.02e+06 pW           9.31e+07 pW           1.34e+08 pW
```

Comparative Analysis

The following table summarizes the improvements achieved by the optimization step.

Metric	Baseline (No CG)	Optimized (With CG)	Improvement
Total Area	5626.24 [um^2]	5019.09 [um^2]	10.8% Reduction
Cell Count	1299	1051	19% Reduction
Total Power	~250 [uW]	~134 [uW]	46.4% Reduction
Clock Network Power	129 nW	24.6 nW	80.9% Reduction
Setup Slack	2.66 ns	3.81 ns	+1.15 ns (Faster)
Hold Slack	0.53 ns	0.53 ns	Stable

Note that we have performed another run of the fusion without clock gating, and without running the compile opt to final command. We noticed a big difference in the setup slack: without the extra optimization the slack is 6.26 ns while with an additional optimization the slack increased to 2.66 ns. We also examine to explain the difference in the slack using the power report but we did not see a difference in the power.

Analysis of Optimization Improvements

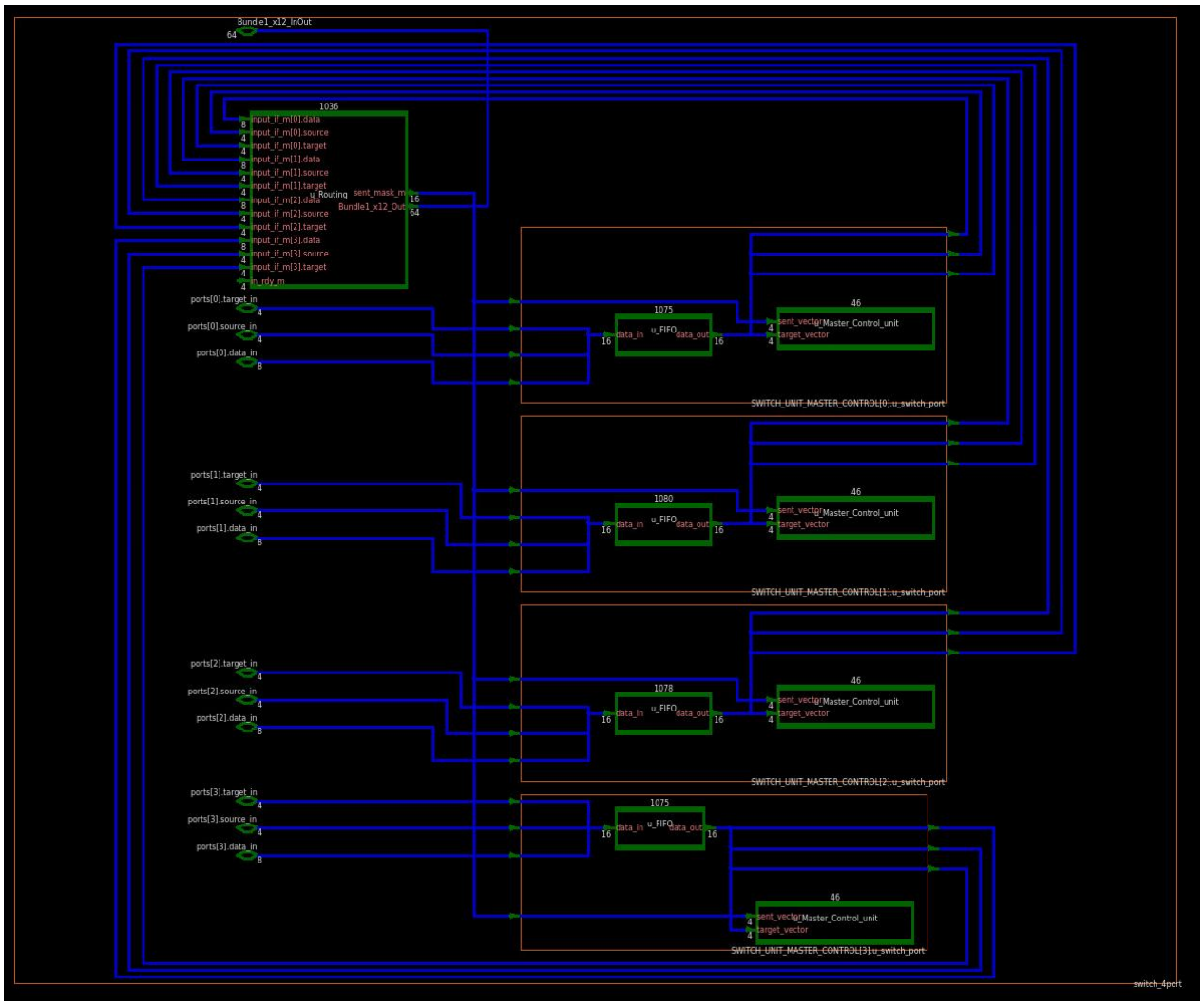
The integration of Clock Gating triggered a systemic improvement across all three key performance metrics.

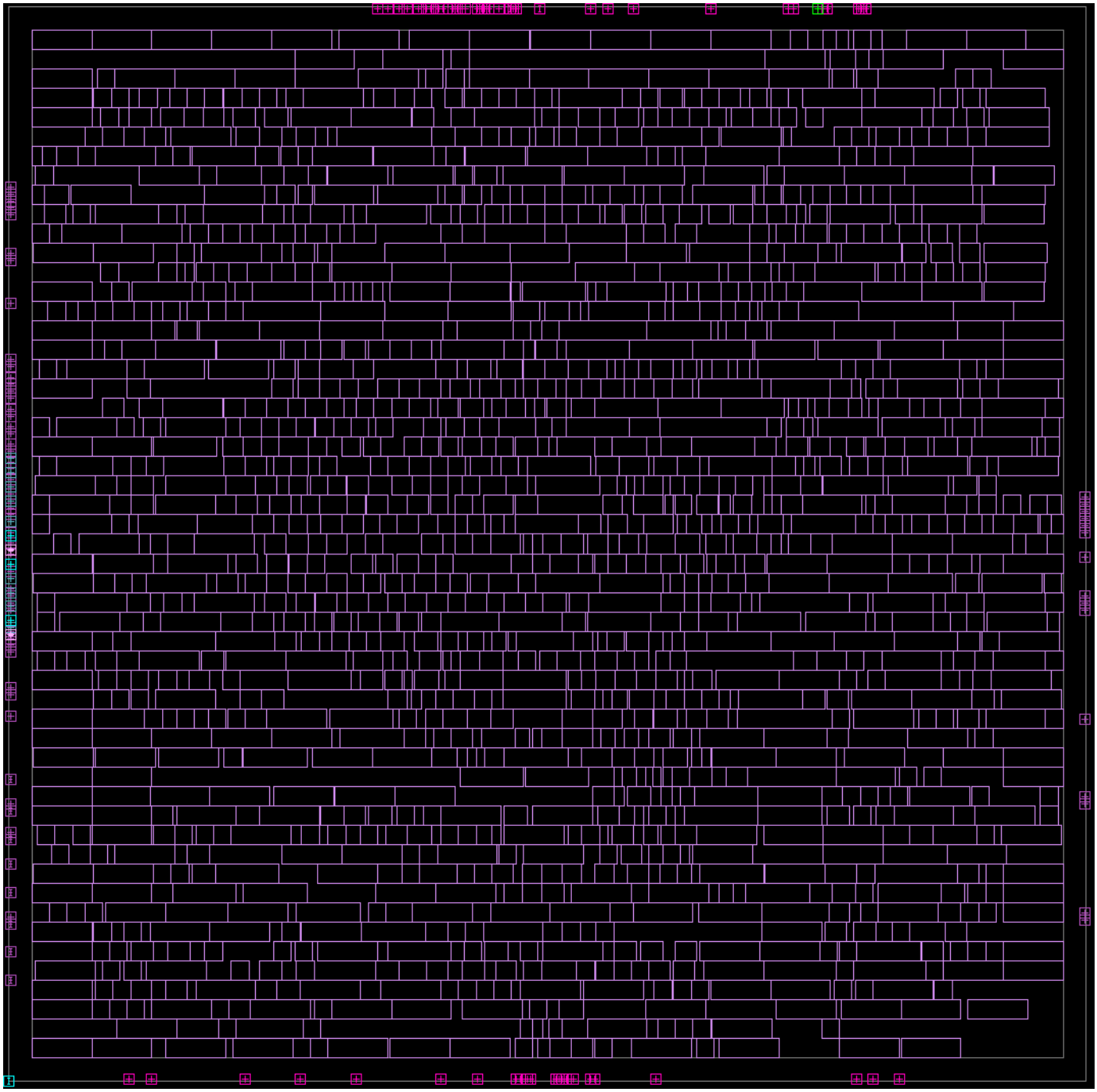
- **Power (46% Reduction):** The most significant gain stems from the Clock Network, where power consumption dropped by ~80% (from 129 nW to 24.6 nW). This was achieved because Integrated Clock Gating (ICG) cells effectively stop the clock signal from toggling at the input of idle registers (particularly within the FIFO buffers), eliminating wasted dynamic switching power.

- **Area (10.8% Reduction):** In the baseline design, registers required external multiplexer feedback loops to retain data when disabled. Enabling Clock Gating allowed the synthesis tool to replace these

bulky feedback loops with compact ICG cells, reducing the total cell count from 1299 to 1051 and significantly shrinking the combinational logic area.

- **Timing (Setup Slack +1.15 ns):** The removal of the feedback multiplexers reduced the logic depth in the critical paths. With fewer gates to traverse and reduced routing congestion due to the smaller overall area, signal propagation delays were shortened, resulting in a substantial increase in Setup Slack (from 2.66 ns to 3.81 ns).





Gate-Level Verification

To ensure that the synthesis and optimization process did not alter the logical functionality of the chip, a Gate-Level Simulation (GLS) was performed.

- **Methodology:** The synthesized netlist (switch_4port.v) and the SDF timing file were verified using the same CRV testbench from Stage B, with adjustments.
- **Result**: The simulation passed with 0 Errors, confirming that the netlist is functionally equivalent to the RTL and ready for layout.

```
checker] OUT0: exp_added=210 act_seen=210 pass=210 fail=0 exp_left=0
[checker] OUT1: exp_added=204 act_seen=204 pass=204 fail=0 exp_left=0
[checker] OUT2: exp_added=197 act_seen=197 pass=197 fail=0 exp_left=0
[checker] OUT3: exp_added=214 act_seen=214 pass=214 fail=0 exp_left=0
$finish called from file "switch_test_GLS.sv", line 187.
$finish at simulation time             10395000
           V C S   S i m u l a t i o n   R e p o r t
Time: 10395000 ps
CPU Time:      0.550 seconds;       Data structure size:   1.4Mb
Sun Jan 18 16:24:36 2026
#qrsh -V -cwd -b y -q sim_normal ./simv +ntb_random_seed_automatic 2>&1 | tee log
```

Performance Analysis: Efficiency & Resource Usage

This section analyzes the performance of both the Verification Environment (Runtime/Memory) and the Hardware Design (Resource Utilization).

Verification Environment Efficiency (Runtime & Memory)

The simulation environment was optimized for performance to ensure fast regression testing. The following metrics were captured from the VCS simulation logs:

- **Throughput:** The regression test successfully routed and verified a total of 825 packets across all output ports.
- **Runtime Analysis:** The complete regression test completed in just 0.550 seconds (CPU Time). This indicates a highly efficient generation and checking mechanism.

- **Memory Usage:** The simulation footprint was extremely low, with a Data structure size of 1.4 Mb. The use of dynamic objects (transactions) was managed efficiently by the SystemVerilog garbage collector, preventing memory leaks.

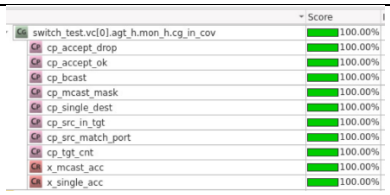
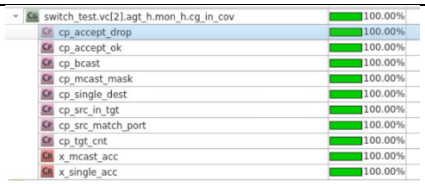
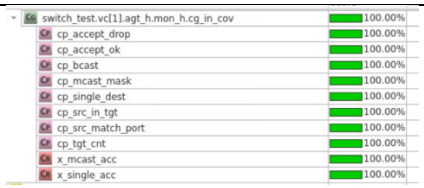
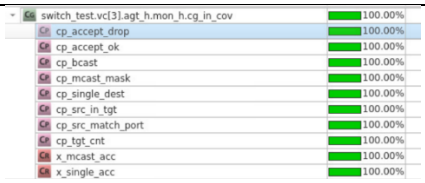
Design Performance (Hardware Efficiency)

In terms of hardware efficiency, the design was analyzed for latency and resource consumption:

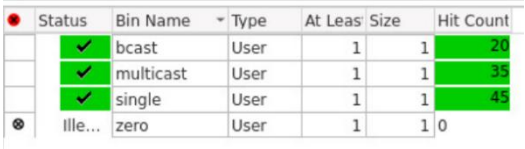
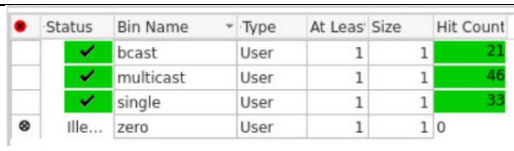
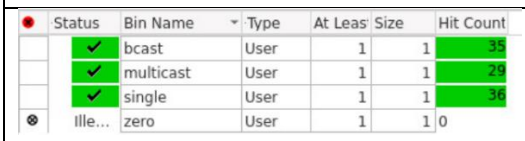
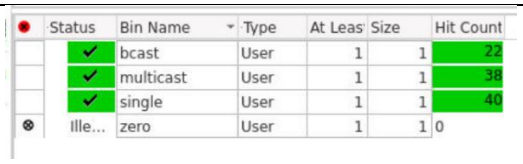
- **Resource Usage:** As detailed earlier, the design utilizes 1051 cells with a total area of 5019 mm^2, demonstrating compact implementation.
- **Bus Utilization:** The Round-Robin arbiter ensures work-conserving behavior, meaning no output port remains idle if there is a pending request, maximizing bus throughput.
- **Memory Efficiency:** The internal FIFOs (Depth=4) were sized to balance between buffering capacity and area/power overhead.

Coverage Appendix

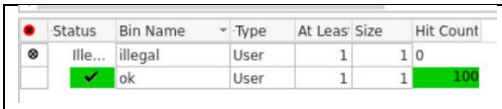
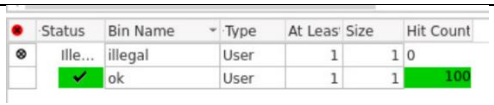
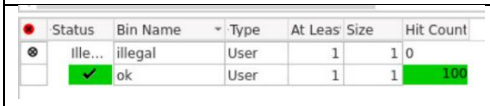
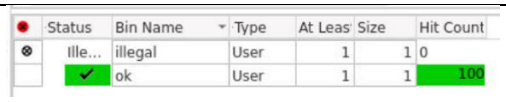
Ingress Coverage (cg_in_cov): Verifies traffic diversity, including cp_src_match_port (integrity check) and cp_mcast_mask (all multicast combinations).

Port0		Port2	
Port1		Port3	

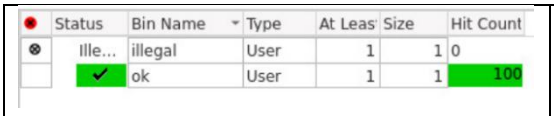
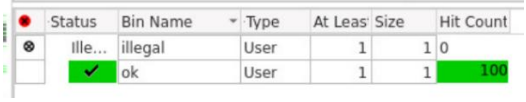
- **cp_tgt_cnt:** Monitors packet classification based on the number of destinations (Single, Multicast, or Broadcast). It includes illegal_bins zero to ensure no packets with an empty target mask were generated.

	Port 0		Port 2
	Port 1		Port 3

- **cp_src_match_port:** An integrity check verifying that the packet's internal source ID matches the physical ingress port ID. It uses illegal_bins to flag any source/port mismatch.

	Port 0		Port 2
	Port 1		Port 3

- **cp_src_in_tgt:** Specifically ensures that for non-broadcast traffic, the source port bit is never set within the target mask, preventing illegal self-routing.

	Port 0		Port 2

Status	Bin Name	Type	At Leas	Size	Hit Count
Ille...	illegal	User	1	1	0
✓	ok	User	1	1	100

Port 1

Status	Bin Name	Type	At Leas	Size	Hit Count
Ille...	illegal	User	1	1	0
✓	ok	User	1	1	100

Port 3

- **cp_accept_drop:** Monitors the Handshake protocol by tracking both successful data acceptance (accepted) and packet discards (drop) due to internal FIFO-full conditions.

Cover Group Item	Definition	Score	U+C	U

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	accepted	User	1	1	93
✓	drop	User	1	1	7

Port 0

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	accepted	User	1	1	89
✓	drop	User	1	1	11

Port 2

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	accepted	User	1	1	87
✓	drop	User	1	1	13

Port 1

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	accepted	User	1	1	91
✓	drop	User	1	1	9

Port 3

- **cp_single_dest:** Confirms that Unicast packets targeted every possible destination port. It utilizes port-specific illegal_bins (e.g., lb_p0 for Port 0) to guarantee no loopback packets were injected.

Status	Bin Name	Type	At Leas	Size	Hit Count
Ille...	d_0001	User	1	1	0
✓	d_0010	User	1	1	12
✓	d_0100	User	1	1	15
✓	d_1000	User	1	1	18
Ille...	lb_p0	User	1	1	0
Ille...	lb_p1	User	1	1	0
Ille...	lb_p2	User	1	1	0
Ille...	lb_p3	User	1	1	0

Port 0

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	d_0001	User	1	1	11
✓	d_0010	User	1	1	7
Ille...	d_0100	User	1	1	0
✓	d_1000	User	1	1	15
Ille...	lb_p0	User	1	1	0
Ille...	lb_p1	User	1	1	0
Ille...	lb_p2	User	1	1	0
Ille...	lb_p3	User	1	1	0

Port 2

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	d_0001	User	1	1	12
Ille...	d_0010	User	1	1	0
✓	d_0100	User	1	1	9
✓	d_1000	User	1	1	15
Ille...	lb_p0	User	1	1	0
Ille...	lb_p1	User	1	1	0
Ille...	lb_p2	User	1	1	0
Ille...	lb_p3	User	1	1	0

Port 1

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	d_0001	User	1	1	14
✓	d_0010	User	1	1	11
✓	d_0100	User	1	1	15
Ille...	d_1000	User	1	1	0
Ille...	lb_p0	User	1	1	0
Ille...	lb_p1	User	1	1	0
Ille...	lb_p2	User	1	1	0
Ille...	lb_p3	User	1	1	0

Port 3

Status	Bin Name	Type	At Leas	Size	Hit Count
Ille...	d_0011	User	1	1	0
Ille...	d_0101	User	1	1	0
✓	d_0110	User	1	1	3
Ille...	d_0111	User	1	1	0
Ille...	d_1001	User	1	1	0
✓	d_1010	User	1	1	7
Ille...	d_1011	User	1	1	0
✓	d_1100	User	1	1	4
Ille...	d_1101	User	1	1	0
✓	d_1110	User	1	1	21
Ille...	lbm_p0	User	1	1	0
Ille...	lbm_p1	User	1	1	0
Ille...	lbm_p2	User	1	1	0

Port 0

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	d_0011	User	1	1	12
Ille...	d_0101	User	1	1	0
Ille...	d_0110	User	1	1	0
Ille...	d_0111	User	1	1	0
✓	d_1001	User	1	1	9
✓	d_1010	User	1	1	4
✓	d_1011	User	1	1	21
Ille...	d_1100	User	1	1	0
Ille...	d_1101	User	1	1	0
Ille...	d_1110	User	1	1	0
Ille...	lbm_p0	User	1	1	0
Ille...	lbm_p1	User	1	1	0

Port 2

	Port 1		Port 3

- **cp_mcast_mask:** Verifies all legal combinations of Multicast groups (2 or 3 destinations). It includes illegal_bins to ensure the source port is never part of the multicast replication mask.

- **cp_bcast:** Specifically monitors the occurrence of Broadcast traffic (4'b1111).

	Port 0		Port 2
	Port 1		Port 3

- **cp_accept_ok:** A helper coverpoint used to filter only successful transactions for cross-coverage analysis.

	Port 0		Port 2
	Port 1		Port 3

- **x_single_acc** & **x_mcast_acc:** Cross-coverage between target masks and the accept signal. This proves that every legal routing combination was not only sent but successfully accepted by the DUT.

	Port 0		Port 2

Status	cp single de	cp accept ol	Type	At Leas	Size	Hit Count
Ille...	[d_0010]	[accepted]	Auto	1	1	0
✓	d_0001	accepted	Auto	1	1	11
✓	d_0100	accepted	Auto	1	1	7
✓	d_1000	accepted	Auto	1	1	14

Status	cp mcast ma	cp accept ol	Type	At Leas	Size	Hit Count
Ille...	[d_0011]	[accepted]	Auto	1	1	0
Ille...	[d_1010 , d_0...	[accepted]	Auto	1	2	0
Ille...	[d_1011 , d_0...	[accepted]	Auto	1	2	0
Ille...	[d_1110]	[accepted]	Auto	1	1	0
✓	d_0101	accepted	Auto	1	1	4
✓	d_1001	accepted	Auto	1	1	2
✓	d_1100	accepted	Auto	1	1	3
✓	d_1101	accepted	Auto	1	1	17

Port 1

Status	cp mcast ma	cp accept ol	Type	At Leas	Size	Hit Count
Ille...	[d_1001]	[accepted]	Auto	1	1	0
Ille...	[d_1100 , d_1...	[accepted]	Auto	1	2	0
Ille...	[d_1110 , d_1...	[accepted]	Auto	1	3	0
✓	d_0011	accepted	Auto	1	1	8
✓	d_0101	accepted	Auto	1	1	3
✓	d_0110	accepted	Auto	1	1	3
✓	d_0111	accepted	Auto	1	1	18

Status	cp single de	cp accept ol	Type	At Leas	Size	Hit Count
Ille...	[d_1000]	[accepted]	Auto	1	1	0
✓	d_0001	accepted	Auto	1	1	14
✓	d_0010	accepted	Auto	1	1	10
✓	d_0100	accepted	Auto	1	1	15

Port 3

Egress (Output) Coverage: cg_out_cov

This group monitors the exit points of the switch to verify data delivery and timing integrity.

switch_test.vc[0].agt_h.mon_h.cg_out_cov 100.00% cp_b2b 100.00% cp_src 100.00% cp_type 100.00% x_type_b2b 100.00%	Port 0
switch_test.vc[2].agt_h.mon_h.cg_out_cov 100.00% cp_b2b 100.00% cp_src 100.00% cp_type 100.00% x_type_b2b 100.00%	Port 2
switch_test.vc[1].agt_h.mon_h.cg_out_cov 100.00% cp_b2b 100.00% cp_src 100.00% cp_type 100.00% x_type_b2b 100.00%	Port 1
switch_test.vc[3].agt_h.mon_h.cg_out_cov 100.00% cp_b2b 100.00% cp_src 100.00% cp_type 100.00% x_type_b2b 100.00%	Port 3

- **cp_type:** Confirms that all packet classifications (Single, Multi, and Broadcast) were physically observed at the output ports.

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	bcast	User	1	1	85
✓	multi	User	1	1	91
✓	single	User	1	1	34

Port 0

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	bcast	User	1	1	85
✓	multi	User	1	1	75
✓	single	User	1	1	37

Port 2

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	bcast	User	1	1	85
✓	multi	User	1	1	92
✓	single	User	1	1	27

Port 1

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	bcast	User	1	1	85
✓	multi	User	1	1	86
✓	single	User	1	1	43

Port 3

- **cp_b2b:** Tracks "Back-to-Back" scenarios where packets are delivered with zero idle cycles between them, testing the DUT's maximum throughput capacity.

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	no	User	1	1	31
✓	yes	User	1	1	179

Port 0

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	no	User	1	1	43
✓	yes	User	1	1	154

Port 2

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	no	User	1	1	32
✓	yes	User	1	1	172

Port 1

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	no	User	1	1	32
✓	yes	User	1	1	182

Port 3

- **cp_src:** Verifies that the environment successfully generated traffic from all four physical ingress ports while utilizing illegal_bins to ensure no invalid one-hot source encodings were processed during simulation.

Status / Bin Name / Type / At Leas / Size / Hit Count Ille... / not_onehot / User / 1 / 1 / 0 ✓ p0 / User / 1 / 1 / 19 ✓ p1 / User / 1 / 1 / 63 ✓ p2 / User / 1 / 1 / 65 ✓ p3 / User / 1 / 1 / 63	Port 0	Status / Bin Name / Type / At Leas / Size / Hit Count Ille... / not_onehot / User / 1 / 1 / 0 ✓ p0 / User / 1 / 1 / 61 ✓ p1 / User / 1 / 1 / 60 ✓ p2 / User / 1 / 1 / 17 ✓ p3 / User / 1 / 1 / 59	Port 2
Status / Bin Name / Type / At Leas / Size / Hit Count Ille... / not_onehot / User / 1 / 1 / 0 ✓ p0 / User / 1 / 1 / 59 ✓ p1 / User / 1 / 1 / 29 ✓ p2 / User / 1 / 1 / 57 ✓ p3 / User / 1 / 1 / 59	Port 1	Status / Bin Name / Type / At Leas / Size / Hit Count Ille... / not_onehot / User / 1 / 1 / 0 ✓ p0 / User / 1 / 1 / 64 ✓ p1 / User / 1 / 1 / 65 ✓ p2 / User / 1 / 1 / 65 ✓ p3 / User / 1 / 1 / 20	Port 3

- **x_type_b2b:** Cross-coverage ensuring that high-speed Back-to-Back scenarios were verified for every individual packet type.

Status / cp type / cp b2b / Type / At Leas / Size / Hit Count ✓ bcast / no / Auto / 1 / 1 / 10 ✓ bcast / yes / Auto / 1 / 1 / 75 ✓ multi / no / Auto / 1 / 1 / 14 ✓ multi / yes / Auto / 1 / 1 / 77 ✓ single / no / Auto / 1 / 1 / 7 ✓ single / yes / Auto / 1 / 1 / 27	Port 0	Status / cp type / cp b2b / Type / At Leas / Size / Hit Count ✓ bcast / no / Auto / 1 / 1 / 21 ✓ bcast / yes / Auto / 1 / 1 / 64 ✓ multi / no / Auto / 1 / 1 / 13 ✓ multi / yes / Auto / 1 / 1 / 62 ✓ single / no / Auto / 1 / 1 / 9 ✓ single / yes / Auto / 1 / 1 / 28	Port 2
Status / cp type / cp b2b / Type / At Leas / Size / Hit Count ✓ bcast / no / Auto / 1 / 1 / 17 ✓ bcast / yes / Auto / 1 / 1 / 68 ✓ multi / no / Auto / 1 / 1 / 13 ✓ multi / yes / Auto / 1 / 1 / 79 ✓ single / no / Auto / 1 / 1 / 2 ✓ single / yes / Auto / 1 / 1 / 25	Port 1	Status / cp type / cp b2b / Type / At Leas / Size / Hit Count ✓ bcast / no / Auto / 1 / 1 / 11 ✓ bcast / yes / Auto / 1 / 1 / 74 ✓ multi / no / Auto / 1 / 1 / 12 ✓ multi / yes / Auto / 1 / 1 / 74 ✓ single / no / Auto / 1 / 1 / 9 ✓ single / yes / Auto / 1 / 1 / 34	Port 3

Internal FSM & White-Box Coverage: cg_fsm_states

Utilizing hierarchical probing, this group monitors the internal logic of the Master Control Unit for each port.

fsm_cvrg[0] 100.00% cp_state 100.00% cp_state_trans 100.00%	Port 0	fsm_cvrg[2] 100.00% cp_state 100.00% cp_state_trans 100.00%	Port 2
fsm_cvrg[1] 100.00% cp_state 100.00% cp_state_trans 100.00%	Port 1	fsm_cvrg[3] 100.00% cp_state 100.00% cp_state_trans 100.00%	Port 3

- **cp_state**: Ensures that the internal controller for each port visited all legal FSM states: IDLE (0), PACKET_RECEIVED (1), and SEE_IF_ALL_PACKET_PASSED (2).

Status / Bin Name / Type / At Leas / Size / Hit Count ✓ IDLE / User / 1 / 1 / 243 ✓ PACKET_RECI... / User / 1 / 1 / 93 ✓ SEE_IF_ALL_P... / User / 1 / 1 / 163 Ille... / bad / User / 1 / 1 / 0	Port 0	Status / Bin Name / Type / At Leas / Size / Hit Count ✓ IDLE / User / 1 / 1 / 252 ✓ PACKET_RECI... / User / 1 / 1 / 89 ✓ SEE_IF_ALL_P... / User / 1 / 1 / 158 Ille... / bad / User / 1 / 1 / 0	Port 2
Status / Bin Name / Type / At Leas / Size / Hit Count ✓ IDLE / User / 1 / 1 / 248 ✓ PACKET_RECI... / User / 1 / 1 / 87 ✓ SEE_IF_ALL_P... / User / 1 / 1 / 164 Ille... / bad / User / 1 / 1 / 0	Port 1	Status / Bin Name / Type / At Leas / Size / Hit Count ✓ IDLE / User / 1 / 1 / 247 ✓ PACKET_RECI... / User / 1 / 1 / 91 ✓ SEE_IF_ALL_P... / User / 1 / 1 / 161 Ille... / bad / User / 1 / 1 / 0	Port 3

- **cp_state_trans**: Monitors the logical transitions between states. It confirms legal sequences (e.g., IDLE to RECEIVED) while using illegal_bins illegal_any_other to ensure no undefined transitions occurred in the RTL.

<table>
<tr>
<td>

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	IDLE	User	1	1	243
✓	PACKET_RECI...	User	1	1	93
✓	SEE_IF_ALL_P...	User	1	1	163
⊗ Ille...	bad	User	1	1	0

</td>
<td>Port 0</td>
<td>

Status	Bin Name	Type	At Leas	Size	Hit Count
⊗ Ille...	illegal_any_ot...	User	1	1	0
✓	legal0:0->1	User	1	1	2
✓	legal0:0[*2]	User	1	1	249
✓	legal1:1->2	User	1	1	89
✓	legal2:2->0	User	1	1	2
✓	legal2:2->1	User	1	1	87
✓	legal2:2[*2]	User	1	1	69

</td>
<td>Port 2</td>
</tr>
<tr>
<td>

Status	Bin Name	Type	At Leas	Size	Hit Count
⊗ Ille...	illegal_any_ot...	User	1	1	0
✓	legal0:0->1	User	1	1	1
✓	legal0:0[*2]	User	1	1	246
✓	legal1:1->2	User	1	1	87
✓	legal2:2->0	User	1	1	1
✓	legal2:2->1	User	1	1	86
✓	legal2:2[*2]	User	1	1	77

</td>
<td>Port 1</td>
<td>

Status	Bin Name	Type	At Leas	Size	Hit Count
⊗ Ille...	illegal_any_ot...	User	1	1	0
✓	legal0:0->1	User	1	1	1
✓	legal0:0[*2]	User	1	1	245
✓	legal1:1->2	User	1	1	91
✓	legal2:2->0	User	1	1	1
✓	legal2:2->1	User	1	1	90
✓	legal2:2[*2]	User	1	1	70

</td>
<td>Port 3</td>
</tr>
</table>

Global System Interaction: cg_global_edge

This group samples system-wide events involving the interaction between multiple ports and internal arbitration logic simultaneously.

- **Ingress Contention (cp_in_cont)**: Tracks whether an input port observes "contention" cycles where multiple requests are made, ensuring the ingress logic can handle high-density traffic transitions.

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	cont	User	1	1	96
✓	no_cont	User	1	1	115

- **Parallel Output Processing (cp_multi_out)**: Tracks clock cycles where more than one output port fired simultaneously, verifying the switch fabric's ability to maintain high throughput through parallel processing.

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	contention	User	1	1	96
✓	no_contention	User	1	1	404

- **Arbiter Stress Analysis (cp_contention)**: A critical metric monitoring "Output Contention"—scenarios where multiple input ports request the same output port in the same cycle, forcing the internal arbiter to resolve the conflict.

Status	Bin Name	Type	At Leas	Size	Hit Count
✓	contention	User	1	1	96
✓	no_contention	User	1	1	404

- **System Stress Cross-Coverage (cx_both)**: Cross-coverage between output load and ingress contention. This creates the ultimate "worst-case" stress scenario for the switch arbitration and routing logic.

Status	cp_multi_out	cp_in_cont	Type	At Leas
✓	multi_out	cont	Auto	1
✓	multi_out	no_cont	Auto	1
✓	single_out	cont	Auto	1
✓	single_out	no_cont	Auto	1